Experiment No: 10

Aim: Explore and implement Autoencoders for image denoising

1. Dataset Source

MNIST Dataset

Source: Built-in dataset in TensorFlow/Keras
Link: https://keras.io/api/datasets/mnist/

2. Dataset Description

The MNIST dataset contains grayscale images of handwritten digits.

- **Number of Instances:** 70,000
- **Image Size:** 28 × 28 pixels

◈ Features:

- Pixel values (0–255)

◈ Target:

- In Autoencoder, input = output
- Model learns to reconstruct clean images

3. Mathematical Formulation

🔷 What is Autoencoder?

Autoencoder is an unsupervised neural network used for:

- Feature learning
- Data compression
- Noise removal

🔷 Architecture

Autoencoder has 3 parts:

1. Encoder

$$\boxed{z = f(x)}$$

2. Latent Space

Bottleneck layer (important for learning features)

3. Decoder

$$\boxed{\hat{x} = g(z)}$$

Reconstructs original input

◆ **Loss Function**

$$\boxed{Loss = ||x - \hat{x}||^2}$$

Minimize difference between:

- Original image
- Reconstructed image

◆ **Denoising Autoencoder**

Input: Noisy image
Output: Clean image

Model learns to:

- Remove noise
- Preserve important features

◆ **Why Autoencoders Work?**

- Learn hidden patterns
- Capture important features
- Ignore noise

4. Algorithm Limitations

- Requires large data
- May learn identity function
- Overfitting risk
- Sensitive to architecture

5. Methodology / Workflow

1. Load MNIST dataset
2. Normalize images
3. Add noise to images
4. Build autoencoder
5. Train model
6. Predict denoised images
7. Visualize results

6. Performance Analysis

- Compare:
  - Noisy images
  - Reconstructed images
- Metrics:
  - Mean Squared Error (MSE)

Better model → cleaner output

7. Hyperparameter Tuning

- Number of layers
- Number of neurons
- Noise level
- Epochs

Code:

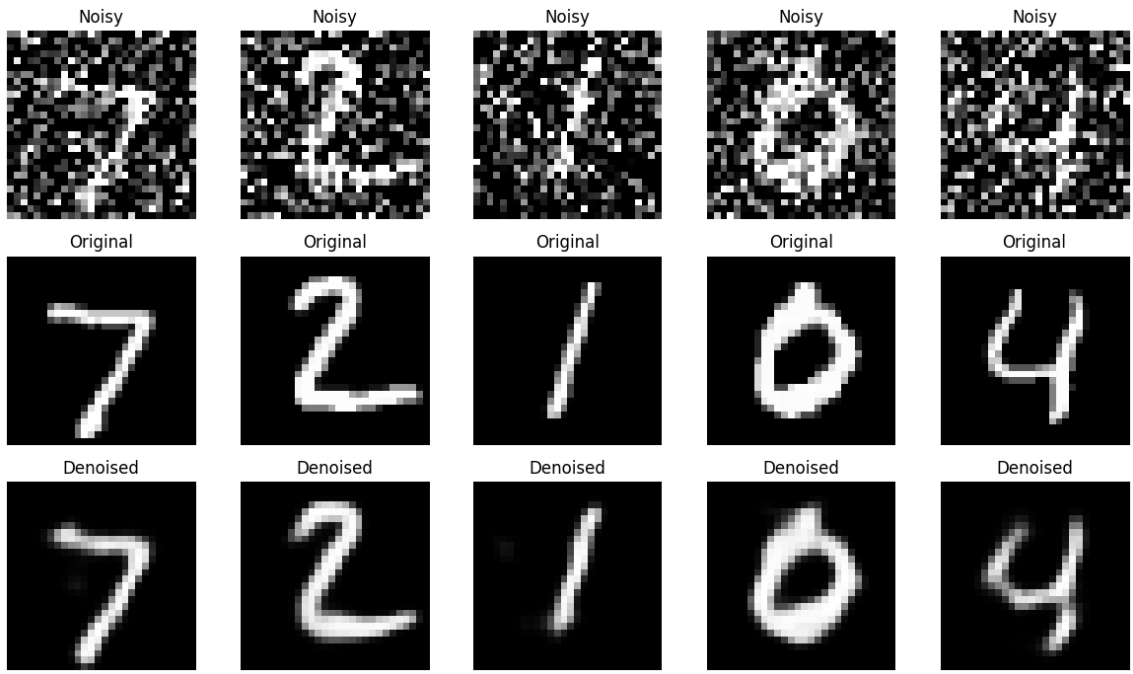
```
# ========================
# 1. Import Libraries
# ========================
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Conv2D, MaxPooling2D, UpSampling2D

# ========================
# 2. Load Dataset
# ========================
(x_train, _), (x_test, _) = mnist.load_data()

# Normalize
x_train = x_train.astype('float32') / 255.
x_test = x_test.astype('float32') / 255.

# Reshape
x_train = x_train.reshape(-1, 28, 28, 1)
x_test = x_test.reshape(-1, 28, 28, 1)
```

```
# ========================
# 3. Add Noise
# ========================
noise_factor = 0.5

x_train_noisy = x_train + noise_factor * np.random.normal(size=x_train.shape)
x_test_noisy = x_test + noise_factor * np.random.normal(size=x_test.shape)

x_train_noisy = np.clip(x_train_noisy, 0., 1.)
x_test_noisy = np.clip(x_test_noisy, 0., 1.)

# ========================
# 4. Autoencoder Model
# ========================
input_img = Input(shape=(28, 28, 1))

# Encoder
x = Conv2D(32, (3,3), activation='relu', padding='same')(input_img)
x = MaxPooling2D((2,2), padding='same')(x)
x = Conv2D(32, (3,3), activation='relu', padding='same')(x)
encoded = MaxPooling2D((2,2), padding='same')(x)

# Decoder
x = Conv2D(32, (3,3), activation='relu', padding='same')(encoded)
x = UpSampling2D((2,2))(x)
x = Conv2D(32, (3,3), activation='relu', padding='same')(x)
x = UpSampling2D((2,2))(x)
decoded = Conv2D(1, (3,3), activation='sigmoid', padding='same')(x)

# Full Autoencoder
autoencoder = Model(input_img, decoded)
autoencoder.compile(optimizer='adam', loss='binary_crossentropy')

# ========================
# 5. Create Encoder Model
# ========================
encoder = Model(input_img, encoded)

# ========================
# 6. Create Decoder Model
# ========================
encoded_input = Input(shape=(7, 7, 32))

decoder_layer = autoencoder.layers[-3](encoded_input)
decoder_layer = autoencoder.layers[-2](decoder_layer)
decoder_output = autoencoder.layers[-1](decoder_layer)

decoder = Model(encoded_input, decoder_output)

# ========================
# 7. Train Model
# ========================
autoencoder.fit(
    x_train_noisy, x_train,
    epochs=5,          # increase to 10 for better result
```

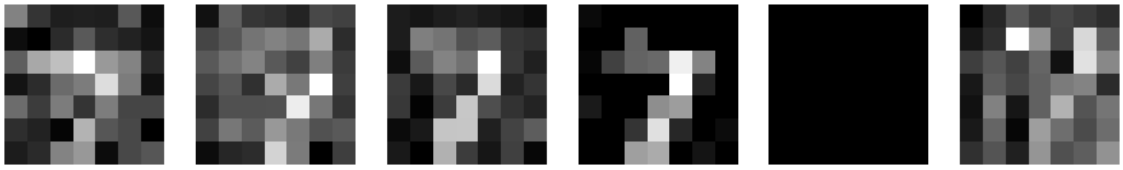
```
    batch_size=128,
    shuffle=True,
    validation_data=(x_test_noisy, x_test)
)

# ========================
# 8. Encode & Decode
# ========================
decoded_imgs = autoencoder.predict(x_test_noisy)
encoded_imgs = encoder.predict(x_test_noisy)
# ========================
# 9. Display Results
# ========================
n = 5
plt.figure(figsize=(12,7))

for i in range(n):
    # Noisy
    plt.subplot(3, n, i+1)
    plt.imshow(x_test_noisy[i].reshape(28,28), cmap='gray')
    plt.title("Noisy")
    plt.axis('off')

    # Original
    plt.subplot(3, n, i+1+n)
    plt.imshow(x_test[i].reshape(28,28), cmap='gray')
    plt.title("Original")
    plt.axis('off')

    # Denoised
    plt.subplot(3, n, i+1+2*n)
    plt.imshow(decoded_imgs[i].reshape(28,28), cmap='gray')
    plt.title("Denoised")
    plt.axis('off')

plt.tight_layout()
plt.show()

# ========================
# 10. Show Encoded Features
# ========================
plt.figure(figsize=(10,4))

for i in range(6):
    plt.subplot(1,6,i+1)
    plt.imshow(encoded_imgs[0][:,:,i], cmap='gray')
    plt.axis('off')

plt.suptitle("Encoded Feature Maps")
plt.show()
```

Output:

```
Epoch 1/5
469/469 ━━━━━━━━━━━━━━━━━━━━ 137s 280ms/step - loss: 0.1719 - val_loss: 0.1189
Epoch 2/5
469/469 ━━━━━━━━━━━━━━━━━━━━ 140s 276ms/step - loss: 0.1147 - val_loss: 0.1106
Epoch 3/5
469/469 ━━━━━━━━━━━━━━━━━━━━ 152s 297ms/step - loss: 0.1086 - val_loss: 0.1055
Epoch 4/5
469/469 ━━━━━━━━━━━━━━━━━━━━ 127s 271ms/step - loss: 0.1057 - val_loss: 0.1035
Epoch 5/5
469/469 ━━━━━━━━━━━━━━━━━━━━ 133s 283ms/step - loss: 0.1037 - val_loss: 0.1018
313/313 ━━━━━━━━━━━━━━━━━━━━ 6s 20ms/step
313/313 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step
```



Encoded Feature Maps



Conclusion:

Autoencoders were successfully implemented for image denoising. The model learned to remove noise from images and reconstruct clean outputs, demonstrating the effectiveness of deep learning in image processing tasks.